

publication-ab-test-methodology

David-Olivier Saban, avec l'aide de Claude Sonnet 4.6 et GPT-5.5

Stop Guessing If Your Agent Config Actually Works

The Problem

Everyone building with coding agents iterates the same way: tweak the system prompt, add a skill, restructure the CLAUDE.md or AGENTS.md, run the agent manually on a task, eyeball the output, decide it “feels better”, commit, move on.

This is how most teams ship prompt and agent configuration changes today. And it’s broken.

There is no measurement. No control group. No way to know if the change that “felt better” on your one manual test actually performs better across the range of tasks your agent handles daily. Worse, there’s no way to catch regressions — a tweak that improves one scenario may silently degrade three others.

The agentic ecosystem is moving fast. New models drop monthly. Skills, hooks, MCP servers, multi-agent orchestration — the configuration surface is exploding. But the evaluation methodology hasn’t kept up. We’re building Formula 1 cars and tuning them by ear.

What We Actually Need

A/B testing isn't new. It's how every serious product team validates changes to user-facing systems. The same discipline applies to agent configurations:

1. **Define what “better” means** — machine-checkable acceptance criteria, not vibes
2. **Isolate variables** — change one thing, hold everything else constant
3. **Run multiple times** — single runs tell you nothing about reliability
4. **Compare against a baseline** — not against your memory of what it used to do

The Methodology

We built a benchmark framework that applies A/B testing rigor to agent configuration validation. Here’s the structure:

Challenges (the “what”)

A challenge is a task definition with: - A prompt (what the agent receives) - Acceptance criteria (independently verifiable assertions — shell commands that exit 0 or 1) - Scoring weights (correctness, efficiency, code quality)

Challenges vary in specificity: some give explicit checklists, others are vague one-liners like “there’s a roundtrip bug, fix it.” This coverage matters — an agent config that only works when hand-held with detailed ACs is not production-ready.

Variants (the “how”)

A variant is an agent configuration under test: - What instructions does the agent see? (full governance, minimal guidance, nothing) - What tools are available? (skills enabled/disabled, hooks active/inactive) - What model runs underneath?

Each variant represents a hypothesis: “this configuration will produce better results than the baseline.”

Isolation

Each run happens in a temporary git worktree. The agent never sees the benchmark infrastructure. Scoring happens after the run, by a separate process that checks acceptance criteria against the agent’s output. The referee is the test suite, not the agent’s self-report.

Scoring

Final score combines: - **Criteria adherence (50%)** — did the agent actually solve the task? - **Efficiency (30%)** — how many tokens and how much time? - **Code quality (20%)** — do existing tests still pass? (regression safety)

The key insight: we track both “Criteria” (average partial completion) and “Perfect” (runs where 100% of criteria passed). An agent that scores 90% criteria but 20% perfect is unreliable — it usually gets close but rarely finishes clean.

A Note On Bias

All results presented here come from a single project: a Node.js monorepo with a well-structured test suite, clear module boundaries, and tasks that range from YAML parser implementation to ambiguous refactoring. The scores, rankings, and deltas are specific to this setup — a different project (different language, different complexity, different test coverage) would produce different numbers.

That said, the structural findings hold regardless of the project: heavy governance degrades autonomous execution, mechanical enforcement outperforms written instructions, newer models are not automatically better, and single manual tests tell you nothing about reliability. These are properties of how agents interact with configuration, not properties of one codebase.

More importantly, the methodology itself is project-agnostic. The value is not in our specific scores — it's in having scores at all. Any team can define their own challenges, their own acceptance criteria, and their own variants. The discipline of measuring before and after is what matters.

What We Found: 150+ Runs Across 6 Model Families

The framework has been exercised across multiple generations of models, multiple CLI tools, and multiple configuration axes. The total dataset spans:

- **115 runs** in the primary baseline (5 variants $\times$ 11 challenges $\times$ multiple iterations)
- **38 runs** in a focused variant comparison (5 variants $\times$ 4 challenges $\times$ 2 model families)
- **Additional exploratory runs** on GPT-5.4 and Opus 4.8 configurations

Models Tested

Model	Tool	Runs	Avg Score	Notes
Claude Opus 4.6 (Bedrock)	Claude Code	20	0.879	Strongest overall performer
Claude Opus 4.6 (API)	Claude Code	54	0.605	Same model, lower score due to challenge mix
Claude Opus 4.7	Claude Code	18	0.627	Newer is not always better on coding benchmarks
Claude Opus 4.8	Claude Code + OpenCode	19+	0.572	Regression vs 4.6 on implementation tasks
Claude Sonnet 4.5	Claude Code	4	0.612	Cheaper, viable under hooks
GPT-5.5	OpenCode	legacy	—	Faster, spontaneously invoked skills
GPT-5.4 (high/medium)	OpenCode	10+	0.000	Total failure — never produces code

The Configuration Winner (38-run focused study)

Variant	Score	Criteria	Tests Pass	Perfect
TDD guidance + hooks	0.872	95%	100%	75%
Bare agent (no config)	0.798	93%	75%	42%

Variant	Score	Criteria	Tests Pass	Perfect
Written	0.696	94%	40%	40%
guidance only				
Hooks only	0.685	80%	60%	40%
Full	0.466	50%	0%	0%
governance				
pipeline				

The Configuration Baseline (115-run production study)

Variant	Runs	Avg Score	Criteria	Perfect Runs
No skills	22	0.669	63%	4
Unguided	22	0.664	60%	6
agent				
No docs	23	0.651	59%	4
Full guidance	25	0.638	58%	6
Minimal	23	0.635	59%	5
guidance				

Delta between variants is thin (0.006 between winner and runner-up, confidence = false). The 115-run study proves a subtler point: **on a well-structured project with 11 diverse challenges, variant choice matters less than model choice and hook enforcement.**

Key Findings

1. Heavy governance kills autonomous agents

The full pipeline (mandatory skills, worktree rules, review steps) scored 50% criteria and 0% test pass. It looks impressive in a CLAUDE.md but blocks the agent from actually coding. The agent spends its budget on process compliance instead of implementation.

2. Written instructions are probabilistic

“Run tests before finishing” in a prompt works ~40% of the time. The agent can ignore it, forget it, or claim it did when it didn’t. Instructions like “follow TDD” produce a measurable behavior shift but not a reliable one.

3. Mechanical enforcement changes the game

Hooks that automatically run tests after edits and block completion when tests fail — this is deterministic. The agent can’t skip it. Under hook enforcement, Sonnet 4.5 (\$0.32/run) achieves the same test-pass rate as Opus 4.6 (\$0.44/run). Hooks normalize safety across model tiers.

4. The winning combination is minimal guidance + mechanical gates

Short, behavior-oriented instructions (TDD approach, small changes) combined with hooks that inject real test failures into context. Score: 0.872, 100% test pass rate, 75% perfect runs.

5. Newer models are not automatically better on coding tasks

Opus 4.8 (0.572) scored lower than Opus 4.6 (0.879 on Bedrock, 0.605 on API) on the same challenges. Opus 4.7 (0.627) fell between them. Without the benchmark, you’d upgrade models assuming improvement. The benchmark catches regressions instantly.

6. Model-tool coupling creates invisible failure modes

GPT-5.4 produces zero code in 300 seconds regardless of configuration. The model spends 100% of its budget on exploration and skill invocation. Removing all configuration files has zero effect — the model exhibits identical behavior whether it receives AGENTS.md, skills, workflows, or nothing at all. This is not a configuration problem. It’s a model behavior pattern invisible without systematic measurement.

7. Content categories have measurable impact

Benchmarking the AGENTS.md file itself reveals three distinct content categories:

Category	Examples	Impact
Operational	Validation commands, workflow, hard rules	+value on all task types
Contextual	Repo structure, module roles, skill list	+value only on ambiguous tasks (+13% score, -34% time)
Noise	Philosophy, strategy, editorial rules	-value everywhere: +25% time, +30% cost, zero benefit

The minimal config (2000 chars of operational + contextual) beats the full config (5000 chars) on clear tasks, while the full config wins on ambiguous ones. The intermediate config — operational + contextual without noise — wins on all three task profiles.

Cross-Model Insights

Model selection matters more than prompt engineering

The spread between the best and worst model (0.879 vs 0.000) dwarfs the spread between the best and worst variant on the same model (0.669 vs 0.635). If you're spending weeks polishing your AGENTS.md but haven't benchmarked model choice, you're optimizing the wrong variable.

Hooks equalize model tiers

Under `e-tdd-hooks`, Opus 4.6 scores 0.903 and Sonnet 4.5 scores 0.862 — a \$0.12/run gap for a 0.041 score difference. Both achieve 100% existing-test pass rate. Without hooks, the gap widens and Sonnet becomes unreliable on regression safety.

Not all models can code autonomously

GPT-5.4 (both high and medium variants) demonstrates that a model can be capable at reasoning yet completely fail at autonomous coding within time constraints. The high variant produces 10-30K reasoning tokens per step but only completes 7-9 steps in 300s — never reaching the implementation phase. The medium variant finishes faster (135s) but writes code in the wrong location, scoring 0% on acceptance criteria.

Without a benchmark, you'd switch models, run it once, see some output, and possibly ship a configuration that literally never produces working code.

How To Use This

The framework is open and pluggable:

```
# Quick single run
```

```
node ab-test/scripts/quick-run.js my-variant --model opus
```

```
# Full matrix: variants × challenges × iterations
```

```
node ab-test/scripts/batch.js --iterations 3 --model opus
```

```
# Compare results
```

```
node ab-test/scripts/report.js
```

The recommended workflow for any agent config change:

1. Run the benchmark on the current config (baseline)
2. Make your change
3. Run the benchmark again
4. Compare scores across all challenges, not just the one you were thinking about

This catches regressions. It catches placebo changes. It catches configurations that work great on structured prompts but fail on vague ones. It catches model upgrades that are actually downgrades.

The Takeaway

If you're iterating on agent configurations — prompts, skills, hooks, tools, model selection — and you're not measuring the results systematically, you're flying blind. You might be making things worse. You probably are, on at least some scenarios.

The evidence from 150+ benchmark runs across six model families is clear:

1. **Measure configuration changes** before committing them. A single manual test tells you nothing about reliability.
2. **Use mechanical enforcement** (hooks, test gates) over written instructions. Prompts are probabilistic; hooks are deterministic.
3. **Benchmark model upgrades**. Newer does not mean better. Opus 4.8 scored lower than Opus 4.6 on our coding tasks.
4. **Keep instructions minimal**. Every unnecessary paragraph dilutes the signal of the rules that actually matter.
5. **Test across task types**. A config that wins on structured prompts may lose on ambiguous ones.

The bar isn't high: define some tasks with checkable criteria, run your agent multiple times, score mechanically, compare variants. That's it. But almost nobody does it.

Stop guessing. Start measuring.